

UNIVERSITE DE YAOUNDÉ I

ÉCOLE NATIONALE SUPÉRIEURE
POLYTECHNIQUE DE YAOUNDÉ

DÉPARTEMENT DE GÉNIES ÉLECTRIQUES
ET DES TÉLÉCOMMUNICATIONS



THE UNIVERSITY OF YAOUNDÉ I

NATIONAL ADVANCED SCHOOL OF
ENGINEERING OF YAOUNDE

DEPARTMENT OF ELECTRICAL AND
TELECOMMUNICATIONS ENGINEERING

GENIE LOGICIEL

GTEL 3046

PRESENTE PAR :

- ▶ **EYEBE SOBGOUN VALERY 22P424 15%**
- ▶ **TONLEU FEUDJIO RAMSES JOEL 22P474 40%**
- ▶ **TIA TCHOFFO BRANDON 23P570 15%**
- ▶ **KENFACK NGEUFACK RICHECARD 23P258 15%**
- ▶ **NGUEKE TSOBENG FRED PAVEL 22P535 15%**

SUPERVISION : Mr. MBIETIEU Amos

ANNÉE ACADÉMIQUE 2025-2026.

ACADEMIC YEAR 2025-2026

REMERCIEMENTS

Avant tout développement, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin de semestre.

Nous adressons en premier lieu nos sincères remerciements à notre professeur superviseur, Monsieur MBIETIEU Amos, pour sa disponibilité, ses conseils avisés et l'encadrement rigoureux qu'il nous a offert tout au long de ce travail. Ses orientations nous ont permis de garder le cap sur les exigences académiques et techniques du projet.

Nous remercions également l'administration de l'École Nationale Supérieure Polytechnique de Yaoundé (ENSPY) pour l'environnement de travail et les ressources pédagogiques mises à notre disposition.

Enfin, nous remercions nos familles et amis pour leur soutien moral et leur compréhension durant les longues nuits de travail que ce projet a nécessitées.

RÉSUMÉ

Ce rapport présente le travail réalisé dans le cadre du projet de Génie Logiciel (GTEL 3046) par le Groupe 7 de la filière GTEL 3 de l'École Nationale Supérieure Polytechnique de Yaoundé. L'objectif principal était de concevoir et de développer une application web de gestion de restaurant, dénommée Restaurant Management System (RMS), en appliquant les méthodologies et outils du génie logiciel moderne.

L'application a été développée avec le framework Django 5.2 (Python 3.10), en utilisant MySQL 8.0 comme système de gestion de base de données et Bootstrap 5 pour l'interface utilisateur. Elle couvre sept modules fonctionnels : l'authentification et la gestion des accès, la gestion des recettes, la gestion du catalogue de produits, la gestion des commandes, la gestion du stock et de l'inventaire, la gestion des ressources humaines et un tableau de bord analytique. Une fonctionnalité bonus a également été ajoutée pour permettre aux clients de passer des commandes sans avoir besoin d'un compte.

Au terme de ce semestre de développement, huit des neuf phases planifiées sont entièrement réalisées, représentant un avancement global de 89%. La phase restante, consacrée aux tests unitaires automatisés et au déploiement en environnement de production, sera finalisée d'ici la fin du semestre.

Mots-clés : *Django, Python, MySQL, Bootstrap, UML, Génie Logiciel, Application Web, Restaurant Management System.*

TABLE DES MATIÈRES

Table des matières

Introduction Générale	8
Organisation du rapport	8
Chapitre 1 — Analyse et Spécification des Besoins	10
1.1 Présentation du domaine d'application	10
1.2 Identification des acteurs du système	10
1.3 Spécification des besoins fonctionnels	11
1.4 Spécification des besoins non fonctionnels	11
Chapitre 2 — Modélisation UML	13
2.1 Diagramme des Cas d'Utilisation	13
2.2 Diagramme de Classes.....	14
2.3 Diagrammes de Séquence.....	15
SD1 — Passage d'une Commande	15
SD2 — Création d'un Utilisateur.....	16
SD3 — Consultation du Dashboard	16
SD4 — Gestion de l'Inventaire.....	16
SD5 — Produits et Recettes	17
2.4 Diagramme de Déploiement.....	17
Chapitre 3 — Architecture et Technologies Utilisées	19
3.1 Architecture MVT de Django.....	19
3.2 Présentation des technologies.....	20
3.3 Configuration de l'environnement de développement	20
Création de l'environnement Anaconda.....	20
Initialisation du projet Django.....	21
Configuration de la connexion MySQL	21
Chapitre 4 — Implémentation des Modules.....	22
4.1 Module 1 — Authentification et Gestion des Accès	22
Modèle User personnalisé	22
Profil employé	22
Vue de connexion	23
4.2 Module 2 — Gestion des Recettes	23
Modèle Recipe et RecipeIngredient	23
4.3 Module 3 — Gestion du Catalogue de Produits	24
Modèle Product avec historique des prix	24
Vue de création de produit avec validation	24

4.4 Module 4 — Gestion des Commandes	25
Transaction atomique pour l'intégrité des données	25
Notification cuisine via Signals	25
Interface cuisine en temps réel	26
4.5 Module 5 — Gestion du Stock et Inventaire	26
Méthode update_quantity avec StockLog	26
4.6 Module 6 — Gestion des Ressources Humaines	27
Rapport mensuel par employé	27
4.7 Module 7 — Tableau de Bord (Dashboard)	27
Cache Django pour l'optimisation des performances	28
Graphiques Chart.js	28
4.8 Fonctionnalité Bonus — Commande Client Publique	28
Chapitre 5 — Tests, Validation et Difficultés	30
5.1 Tests manuels effectués	30
5.2 Difficultés rencontrées et solutions apportées	31
Chapitre 6 — Résultats et Présentation de l'Application	32
6.1 État de l'application	32
6.2 Captures d'écran des interfaces	33
Page d'accueil	33
Tableau de Bord	33
Interface Commandes et Cuisine	34
Commande Client Publique	35
6.3 Planning GANTT — Avancement du projet	35
6.4 Dépôt GitHub et versionnement	36
Conclusion Générale et Perspectives	38
Perspectives	38
Bibliographie	40
Documentation Officielle	40
Ouvrages de Référence	40
Cours et Supports Pédagogiques	40
Ressources en Ligne	40

LISTE DES FIGURES

Figure	Description	Page
Figure 1	Diagramme des Cas d'Utilisation — RMS Groupe 7	8
Figure 2	Extrait du Diagramme de Classes — principales associations	10
Figure 3	SD1 — Séquence de Passage d'une Commande	12
Figure 4	SD4 — Séquence de Gestion de l'Inventaire Stock	14
Figure 5	Diagramme de Déploiement — Architecture 5 nœuds	16
Figure 6	Architecture MVT Django du projet RMS	18
Figure 7	Page d'accueil — Restaurant Groupe 7	44
Figure 8	Tableau de bord avec KPIs et graphiques Chart.js	44
Figure 9	Interface Nouvelle Commande	45
Figure 10	Interface Cuisine en temps réel	45
Figure 11	Commande Client Publique sans connexion	45
Figure 12	Diagramme de GANTT — Avancement Semaine 17	46

LISTE DES TABLEAUX

Tableau	Description	Page
Tableau 1	Récapitulatif des acteurs et leurs rôles	4
Tableau 2	Besoins fonctionnels par module	5
Tableau 3	Technologies et versions utilisées	18
Tableau 4	Liste des 15 modèles Django et leurs méthodes	22
Tableau 5	URLs fonctionnelles de l'application	43
Tableau 6	Résultats des tests manuels effectués	39

Tableau	Description	Page
Tableau 7	Difficultés rencontrées et solutions	41
Tableau 8	Avancement global par phase	46

Introduction Générale

Dans un contexte économique où la transformation numérique touche progressivement tous les secteurs d'activité, la restauration ne fait pas exception. Les établissements de restauration modernes font face à des défis croissants en matière de gestion : coordination entre la cuisine et la salle, suivi des stocks en temps réel, gestion du personnel, analyse des performances financières. Des solutions logicielles adaptées s'avèrent indispensables pour répondre à ces enjeux.

C'est dans ce contexte que s'inscrit notre projet de Génie Logiciel, initié dans le cadre du cours GTEL 3046 à l'École Nationale Supérieure Polytechnique de Yaoundé (ENSPY). Nous avons été chargés, en tant que Groupe 7 de la filière GTEL 3, de concevoir et de développer une application web complète de gestion de restaurant, désignée sous l'acronyme RMS (Restaurant Management System).

Ce projet nous a permis de mettre en pratique l'ensemble des concepts théoriques abordés dans le cours : la modélisation orientée objet avec UML, la gestion du cycle de vie logiciel, la conception d'architectures web, la gestion de bases de données relationnelles et la programmation orientée objet en Python. Au-delà des aspects techniques, il nous a également confrontés aux réalités du travail en équipe sur un projet d'envergure : répartition des tâches, coordination, suivi d'avancement et respect des délais.

Le présent rapport constitue la documentation complète de notre travail. Il retrace toutes les étapes du projet, de l'analyse initiale des besoins jusqu'à la livraison de l'application fonctionnelle, en passant par la modélisation UML, les choix d'architecture, l'implémentation des modules et les tests de validation.

Organisation du rapport

Ce rapport est structuré en six chapitres :

- Le Chapitre 1 présente l'analyse et la spécification des besoins du système, avec l'identification des acteurs et des fonctionnalités attendues.
- Le Chapitre 2 décrit la modélisation UML complète : diagramme des cas d'utilisation, diagramme de classes, diagrammes de séquence et diagramme de déploiement.
- Le Chapitre 3 expose l'architecture technique retenue et les technologies utilisées.

- Le Chapitre 4 détaille l'implémentation des sept modules fonctionnels de l'application.
- Le Chapitre 5 présente les tests effectués et les difficultés rencontrées durant le développement.
- Le Chapitre 6 propose une vue d'ensemble des résultats obtenus avec les captures de l'application finale.

Chapitre 1 — Analyse et Spécification des Besoins

1.1 Présentation du domaine d'application

Un restaurant est un établissement commercial dont le fonctionnement quotidien implique la coordination de plusieurs activités simultanées : la prise de commande auprès des clients, la préparation des plats en cuisine, la gestion des approvisionnements et des stocks, la gestion du personnel de salle et de cuisine, ainsi que le suivi des performances financières.

Dans un restaurant de taille moyenne, sans outil informatique adapté, la communication entre la salle et la cuisine repose souvent sur des bons de commande manuscrits, source d'erreurs et de retards. La gestion des stocks est réalisée de façon empirique, sans visibilité en temps réel sur les niveaux de disponibilité. Le suivi financier, quant à lui, nécessite un travail de consolidation manuel qui est à la fois chronophage et propice aux erreurs.

Notre application RMS a pour vocation de centraliser et d'automatiser ces différentes activités au sein d'un système d'information unique, accessible via un navigateur web et adapté à différents profils d'utilisateurs.

1.2 Identification des acteurs du système

À partir de notre étude du domaine et des discussions avec notre superviseur, nous avons identifié six catégories d'acteurs interagissant avec le système. Chaque acteur dispose de droits d'accès spécifiques, définis selon le principe du moindre privilège.

Acteur	Rôle dans le restaurant	Modules accessibles	Droits
Administrateur	Responsable du système informatique	Tous les modules (M1 à M7) + Admin Django	Lecture + Écriture + Suppression
Directeur	Responsable de l'établissement	M7 Dashboard, M4 Commandes	Lecture + Rapports
Head Chef	Chef de cuisine responsable	M2 Recettes, M3 Produits, M4 Cuisine	Lecture + Écriture recettes
Cook (Cuisinier)	Personnel de cuisine	M4 Interface Cuisine uniquement	Lecture + Mise à jour statuts

Acteur	Rôle dans le restaurant	Modules accessibles	Droits
Server (Serveur)	Personnel de salle	M4 Commandes, M3 Menu	Lecture + Création commandes
Stock Manager	Gestionnaire des approvisionnements	M5 Stock et Inventaire	Lecture + Écriture stock
Client	Client du restaurant	Page Menu + Commande Publique	Création commande uniquement

Tableau 1 — Récapitulatif des acteurs et de leurs droits d'accès

1.3 Spécification des besoins fonctionnels

À partir de l'analyse du domaine et des acteurs identifiés, nous avons défini les besoins fonctionnels regroupés en sept modules. Chaque module correspond à une dimension spécifique de la gestion du restaurant.

Module	Besoins fonctionnels principaux
M1 — Authentification	Gestion des rôles et permissions, connexion/déconnexion sécurisée, création de comptes utilisateurs, profils employés
M2 — Recettes	Création et modification des recettes, association ingrédients-stock, vérification disponibilité, calcul coût de revient
M3 — Produits	Catalogue de plats par catégories, gestion des prix avec historique, images des plats, menu public accessible sans connexion
M4 — Commandes	Prise de commande avec transaction atomique, interface cuisine temps réel, suivi des statuts, gestion des tables, livraison
M5 — Stock	Inventaire en temps réel, alertes automatiques sur seuils critiques, historique des variations (StockLog), déduction automatique
M6 — RH	Dossiers des employés, affectation par département, calcul salaire net, rapport mensuel par employé
M7 — Dashboard	KPIs financiers (CA, commandes), graphiques Chart.js, best-sellers, cache pour optimisation, export de rapports

Tableau 2 — Besoins fonctionnels par module

1.4 Spécification des besoins non fonctionnels

Au-delà des fonctionnalités métier, nous avons identifié plusieurs exigences non fonctionnelles qui ont guidé nos choix de conception.

Sécurité : Toutes les pages de gestion doivent nécessiter une authentification. Les mots de passe sont stockés sous forme hachée (algorithme Django PBKDF2-SHA256). La protection CSRF est activée sur tous les formulaires.

Performance : Les pages à fort trafic (dashboard) doivent répondre en moins d'une seconde grâce à la mise en cache. Les requêtes de base de données doivent être optimisées par l'utilisation de `select_related()` et `prefetch_related()`.

Disponibilité : L'application doit fonctionner en local sans connexion internet (Bootstrap servi en local), avec basculement automatique vers le CDN si disponible.

Maintenabilité : Le code doit suivre les conventions Django (MVT), être versionné sur GitHub et documenté par des commentaires clairs.

Compatibilité : L'interface doit être responsive et fonctionner sur Chrome, Firefox et Edge.

Chapitre 2 — Modélisation UML

La phase de modélisation a occupé les cinq premières semaines de notre projet, conformément au diagramme de GANTT établi. Nous avons produit sept diagrammes UML couvrant les différentes perspectives du système : fonctionnelle (DCU), structurelle (classes) et dynamique (séquences, déploiement).

2.1 Diagramme des Cas d'Utilisation

Le diagramme des cas d'utilisation représente les interactions entre les acteurs externes et le système. Il constitue le point de départ de toute la modélisation, car il nous permet de capturer les fonctionnalités attendues du point de vue des utilisateurs, sans entrer dans les détails techniques d'implémentation.

Notre DCU identifie sept acteurs (dont le Client qui est un acteur secondaire ajouté lors du développement pour la fonctionnalité de commande publique) interagissant avec les sept modules fonctionnels. Nous avons utilisé les relations «include» pour les fonctionnalités obligatoires (comme la vérification du stock lors de la création d'une commande) et les relations «extend» pour les fonctionnalités optionnelles (comme l'ajout d'une livraison).

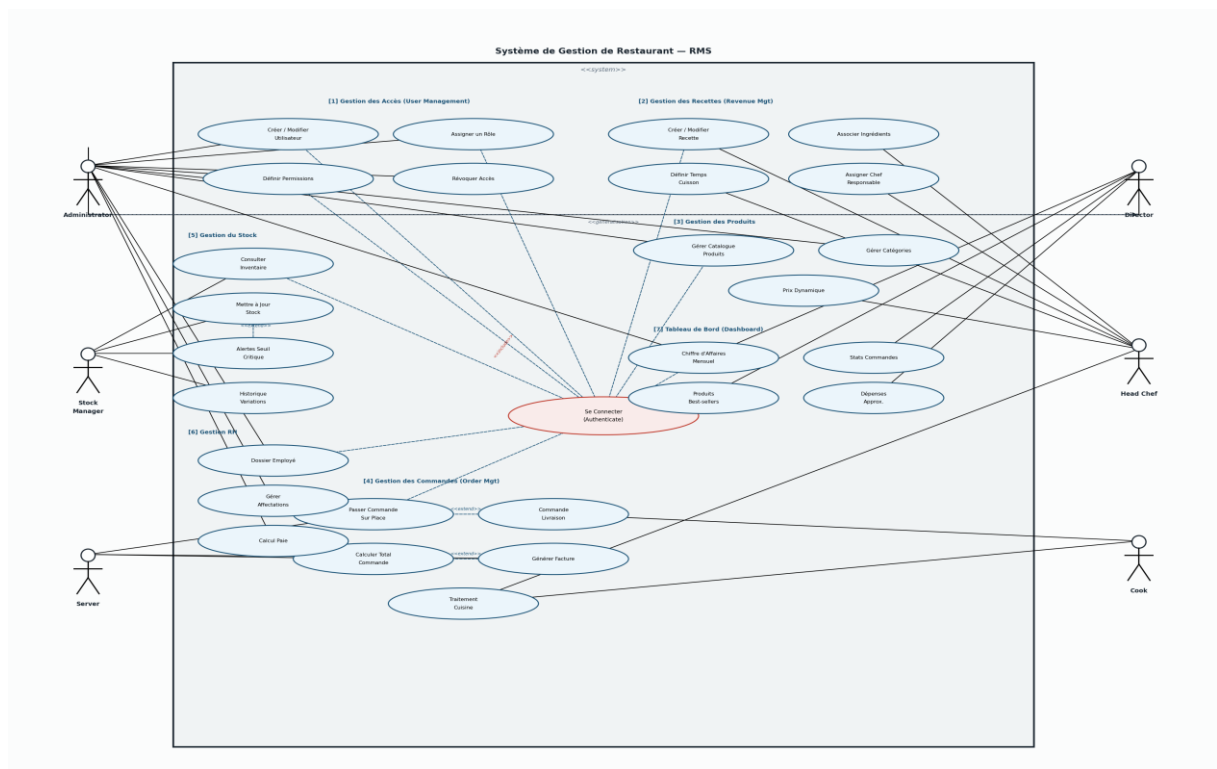


Figure 1 — Diagramme des Cas d'Utilisation — RMS Groupe 7 (7 modules, 6 acteurs)

2.2 Diagramme de Classes

Le diagramme de classes décrit la structure statique du système. Il modélise les classes, leurs attributs, leurs méthodes et les relations qui les unissent. Notre système comprend quinze classes principales, dont plusieurs héritent de classes Django existantes (AbstractUser pour la classe User).

Nous avons accordé une attention particulière aux multiplicités et aux types de relations : la relation entre Order et OrderLine est une composition (une OrderLine ne peut pas exister sans son Order parent), tandis que la relation entre Product et Stock est une association simple (un produit peut exister sans être lié à un stock particulier).

Voici les principales classes de notre modèle de données :

Classe Django	Héritage / Relations	Attributs principaux	Méthodes clés
Role	—	role_name, description	__str__()
User	Hérite de AbstractUser	role (FK vers Role)	has_module_access()
EmployeeProfile	OneToOne avec User	salary, hire_date, department	calculate_payroll()
Category	—	label, description, order	__str__()
Product	FK vers Category	name, price, is_available, image	toggle_availability(), update_price()
PriceHistory	FK vers Product	old_price, new_price, changed_at	__str__()
Stock	FK vers Product	item_name, quantity, unit, threshold	update_quantity(), trigger_alert()
StockLog	FK vers Stock	old_qty, new_qty, delta, updated_at	__str__()
Recipe	OneToOne avec Product	cooking_time, instructions, chef	check_all_ingredients_available()
RecipeIngredient	FK vers Recipe + Stock	required_qty, unit	check_availability(), deduct_from_stock()
Table	—	number, capacity, status	is_available(), release()
Order	FK vers Table + User	date, status, total_amount, order_type	calculate_total(), notify_kitchen()
OrderLine	FK vers Order + Product	quantity, unit_price, subtotal	save() — auto-calcul subtotal

Classe Django	Héritage / Relations	Attributs principaux	Méthodes clés
Delivery	OneToOne avec Order	address, status, driver, delivery_time	confirm_delivery()
Revenue	—	date, total_orders, gross_amount	get_best_sellers() (classmethod)

Tableau 3 — Liste des 15 modèles Django avec leurs méthodes

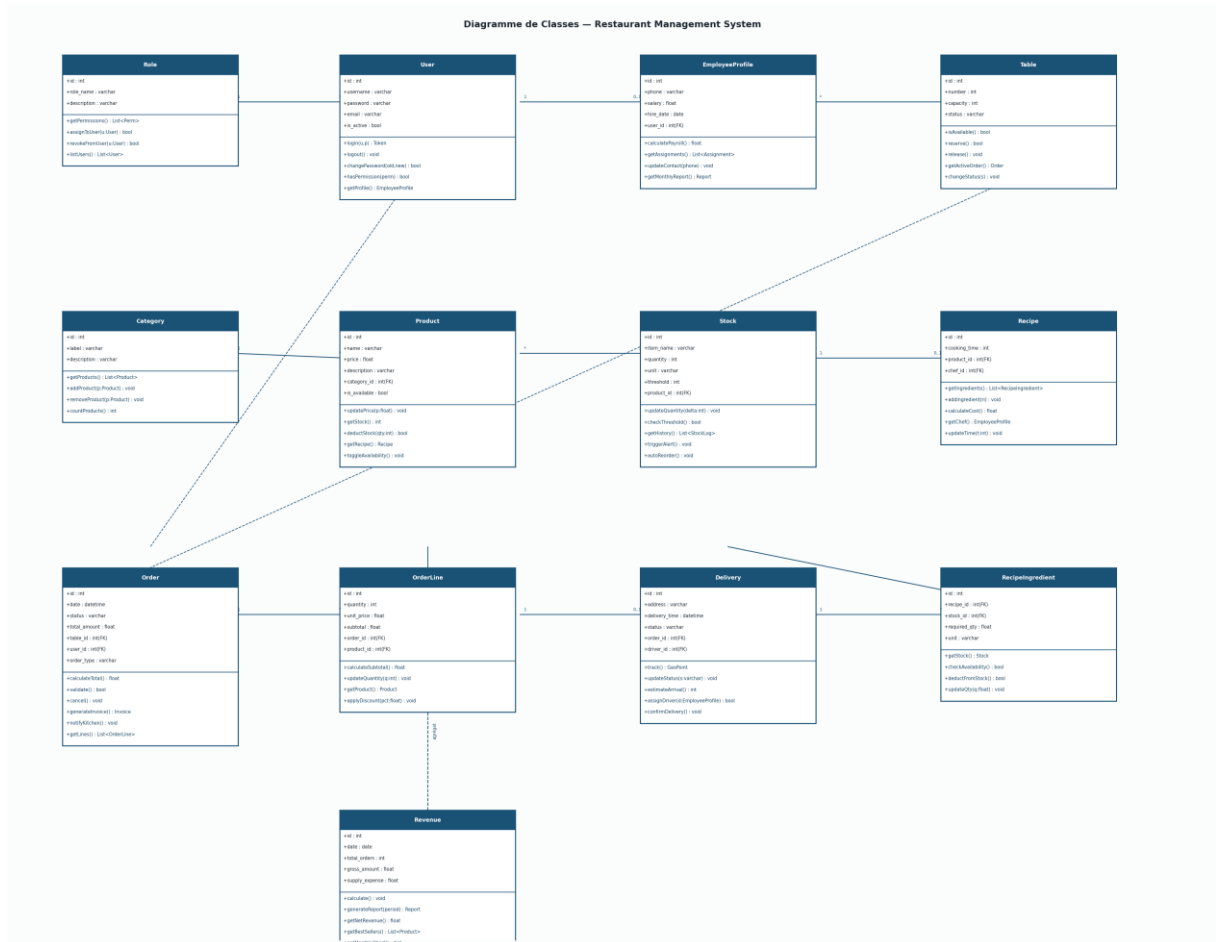


Figure 2 — Extrait du Diagramme de Classes — associations entre les principales entités

2.3 Diagrammes de Séquence

Les diagrammes de séquence illustrent le comportement dynamique du système en décrivant les interactions entre les objets dans le temps. Nous en avons produit cinq, couvrant les scénarios les plus complexes et les plus représentatifs du système.

SD1 — Passage d'une Commande

Ce diagramme modélise le scénario central du système : la création d'une commande par un serveur. Il comprend plusieurs fragments ALT et OPT pour couvrir les cas nominaux et alternatifs. Le fragment ALT principal gère la disponibilité du stock — si un produit est en

rupture, la commande est annulée avec ROLLBACK. Un fragment OPT gère l'association optionnelle à une table.

Fragments UML utilisés — SD1 : ALT [stock disponible / rupture de stock], OPT [commande sur table], @transaction.atomic garantissant l'atomicité.

SD2 — Création d'un Utilisateur

Ce diagramme décrit le processus de création d'un nouveau compte employé par l'administrateur. Les fragments LOOP gèrent la répétition en cas de données invalides (nom d'utilisateur déjà pris, mots de passe non concordants). Le fragment OPT gère l'envoi optionnel d'un email de bienvenue.

SD3 — Consultation du Dashboard

Ce diagramme montre comment le tableau de bord exploite le cache Django pour optimiser les performances. Le fragment ALT distingue le cas où les données sont en cache (retour immédiat) du cas où elles doivent être recalculées depuis la base de données. Le fragment OPT gère l'export optionnel des données en PDF ou Excel.

SD4 — Gestion de l'Inventaire

Ce diagramme décrit la mise à jour d'un article de stock par le gestionnaire. Après la modification de la quantité, le système vérifie si le seuil critique est atteint. Si oui, une alerte email est envoyée automatiquement. Un historique (StockLog) est créé à chaque modification, quelle que soit la quantité.

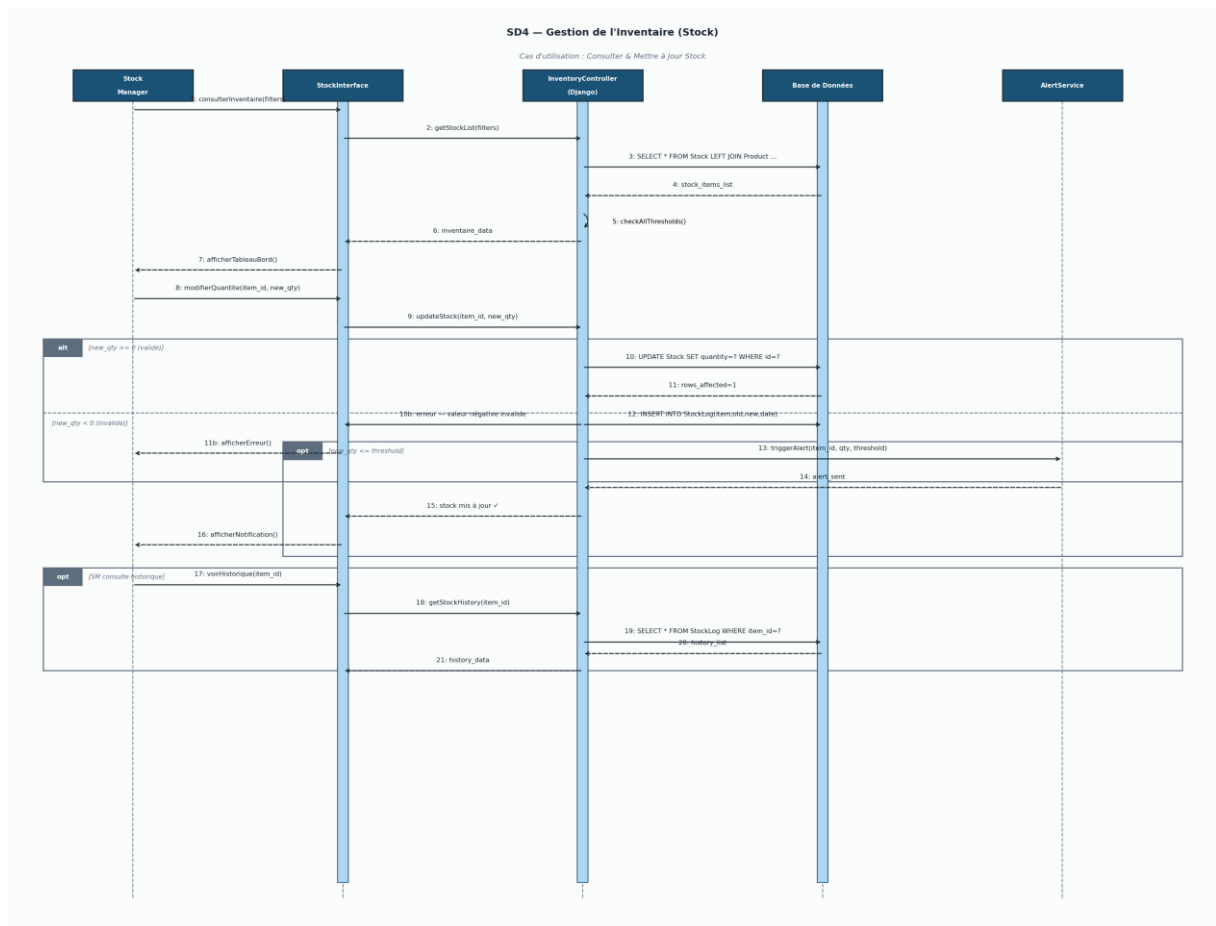


Figure 4 — SD4 : Gestion de l'Inventaire avec fragments ALT (seuil) et OPT (alerte email)

SD5 — Produits et Recettes

Ce dernier diagramme de séquence couvre la gestion du catalogue produits et des recettes. Il comprend un fragment OPT pour l'association optionnelle d'une recette à un plat, et un fragment ALT pour la gestion de la validation du prix (prix nul ou négatif rejeté avec message d'erreur).

2.4 Diagramme de Déploiement

Le diagramme de déploiement décrit l'architecture physique de déploiement du système, c'est-à-dire les nœuds matériels et logiciels ainsi que les artefacts qui y sont déployés. Notre architecture cible pour la mise en production comprend cinq nœuds principaux.

En développement (configuration actuelle), l'ensemble de l'application tourne sur la machine locale du développeur, avec Django comme serveur de développement intégré. Pour la mise en production, nous prévoyons de déployer sur un serveur Ubuntu avec Gunicorn comme serveur WSGI et Nginx comme proxy inverse, conformément aux meilleures pratiques Django.

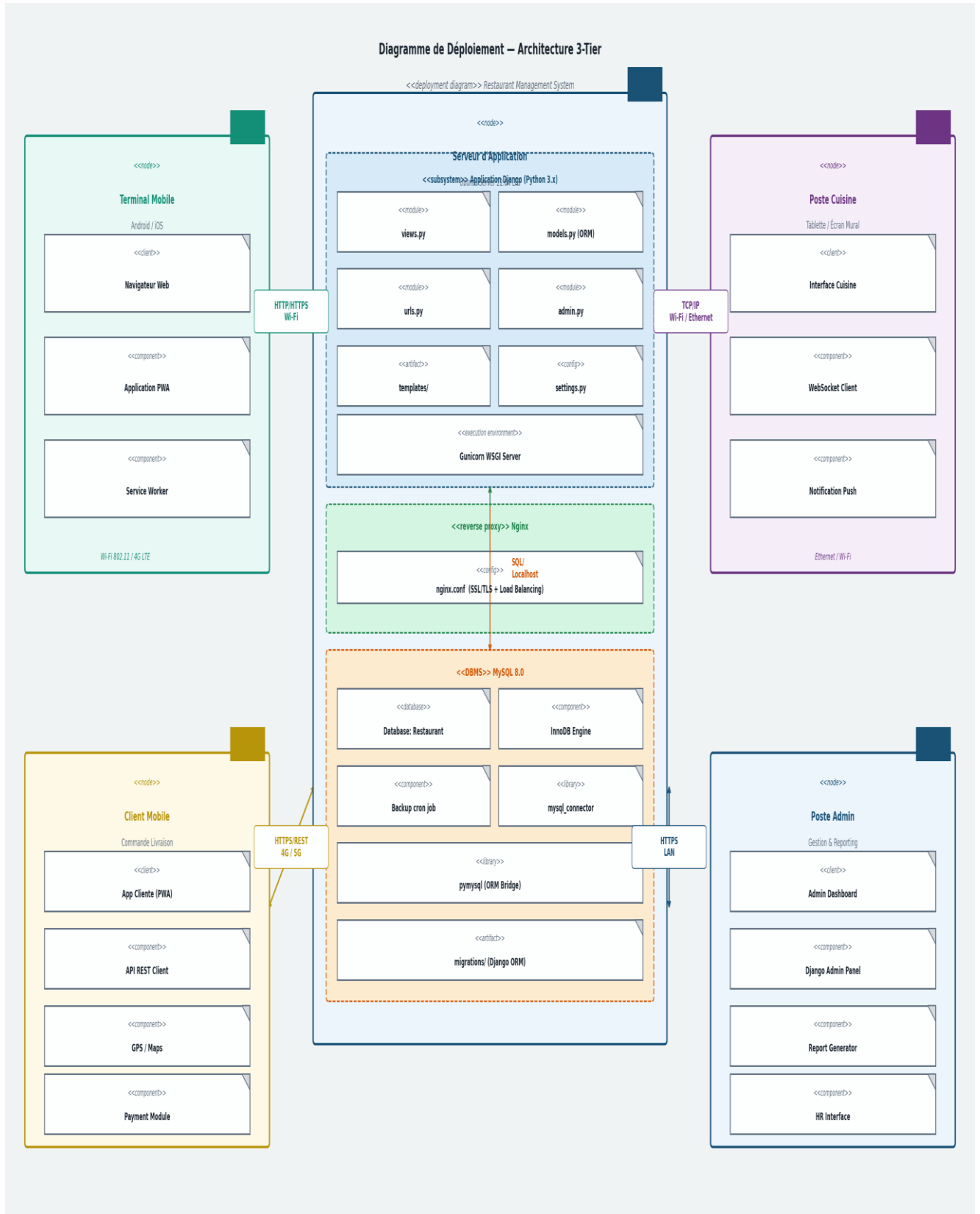


Figure 5 — Diagramme de Déploiement : architecture 5 nœuds (dev local + cible production)

Chapitre 3 — Architecture et Technologies Utilisées

3.1 Architecture MVT de Django

Django impose une architecture Modèle-Vue-Template (MVT), variante de l'architecture MVC (Modèle-Vue-Contrôleur) classique. Cette séparation des responsabilités est fondamentale dans notre conception car elle facilite la maintenance, les tests et l'évolution de l'application.

Dans notre projet, cette architecture se concrétise de la façon suivante :

- Les Modèles (models.py) : définissent la structure des données et la logique métier. Ils correspondent aux tables de la base de données MySQL et encapsulent toute la logique de manipulation des données (calculs, validations, alertes).
- Les Vues (views.py) : constituent le point d'entrée des requêtes HTTP. Elles récupèrent les données via les modèles, appliquent les règles métier et transmettent le contexte aux templates. Nous avons utilisé des vues basées sur des fonctions (Function-Based Views) pour leur lisibilité et leur flexibilité.
- Les Templates (HTML + Bootstrap) : gèrent l'affichage. Ils reçoivent le contexte des vues et génèrent les pages HTML envoyées au navigateur. Nous avons utilisé le système de template Django avec l'héritage (`{% extends %}`) pour éviter la duplication de code.
- Les URLs (urls.py) : assurent le routage entre les URLs demandées et les vues correspondantes. Le fichier principal inclut les URLs de l'application via `include()`.

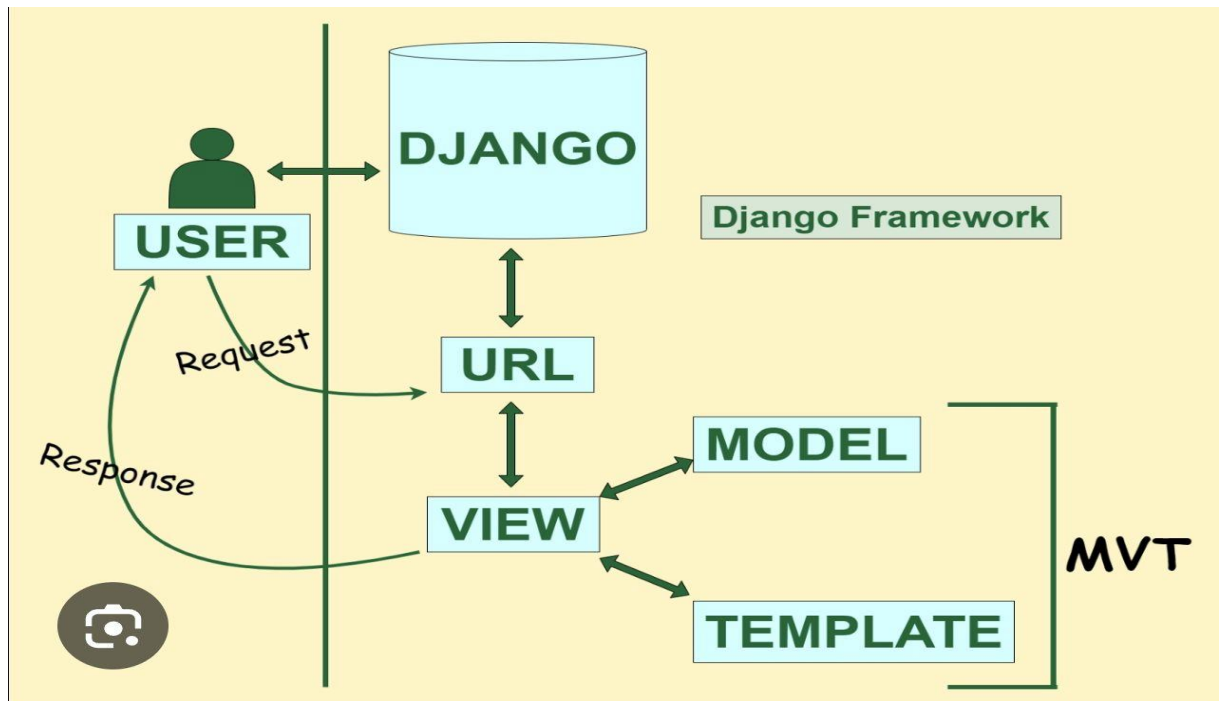


Figure 6 — Architecture MVT de Django appliquée au projet RMS

3.2 Présentation des technologies

Technologie	Version	Rôle dans le projet	Justification du choix
Python	3.10	Langage de programmation principal	Langage imposé par le cours, polyvalent et lisible
Django	5.2.14	Framework web backend (MVT)	Framework Python le plus mature pour le web, ORM puissant
MySQL	8.0	Système de gestion de base de données	SGBD relationnel robuste, compatible Django via pymysql
pymysql	1.2.0	Connecteur Python-MySQL	Compatibilité Windows (contrairement à mysqlclient)
Bootstrap	5.3.0	Framework CSS frontend	Composants UI prêts à l'emploi, responsive design
Chart.js	4.4.0	Bibliothèque graphiques JavaScript	Légère, facile à intégrer avec Django JSON, nombreux types
Bootstrap Icons	1.11.0	Bibliothèque d'icônes SVG	Cohérence avec Bootstrap, plus de 1800 icônes disponibles
Pillow	12.2.0	Traitement des images Python	Nécessaire pour ImageField Django (photos des plats)
Anaconda	Python 3.10	Gestionnaire d'environnements Python	Isolation des dépendances, gestion simplifiée sur Windows
PyCharm Community	Latest	IDE de développement	Débogage intégré, inspection de code, terminal intégré
Git & GitHub	Latest	Versionnement du code source	Collaboration en équipe, historique, sauvegarde distante

Tableau 4 — Technologies et versions utilisées dans le projet

3.3 Configuration de l'environnement de développement

La mise en place de l'environnement de développement a nécessité une démarche rigoureuse, que nous décrivons ici car elle a été source de plusieurs difficultés initiales.

Création de l'environnement Anaconda

Nous avons commencé par créer un environnement virtuel Python dédié au projet afin d'isoler nos dépendances des autres projets de la machine. L'environnement a été nommé `resto_env7` en référence à notre groupe.

```
conda create -n resto_env7 python=3.10
conda activate resto_env7
pip install django pymysql Pillow django-crispy-forms crispy-bootstrap5
```

Initialisation du projet Django

La création du projet Django requiert une attention particulière sur la syntaxe de la commande. Nous avons initialement oublié le point (.) final, ce qui a créé une structure de dossiers imbriqués incorrecte. La commande correcte est :

```
cd C:\Users\ELITEBOOK MT45\Desktop
mkdir resto_groupe7 && cd resto_groupe7
django-admin startproject resto_groupe7 .    # Le point est indispensable
python manage.py startapp restaurant
```

Configuration de la connexion MySQL

La connexion entre Django et MySQL a nécessité deux configurations spécifiques dans le fichier `settings.py`. Premièrement, l'import de `pymysql` doit être effectué en toute première ligne du fichier, avant tout import Django, sinon le driver n'est pas reconnu sur Windows. Deuxièmement, le paramètre `AUTH_USER_MODEL` doit être défini avant la première migration pour que Django reconnaisse notre modèle User personnalisé.

```
# settings.py — Ligne 1 absolument
import pymysql
pymysql.install_as_MySQLdb()

# Plus bas dans settings.py
AUTH_USER_MODEL = "restaurant.User" # AVANT toute migration

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.mysql",
        "NAME": "Restaurant",
        "USER": "root",
        "PASSWORD": "votre_mot_de_passe",
        "HOST": "127.0.0.1",
        "PORT": "3306",
        "OPTIONS": {"charset": "utf8mb4"},
    }
}
```

Chapitre 4 — Implémentation des Modules

Ce chapitre présente le cœur du travail de développement. Nous y décrivons, pour chacun des sept modules, les choix d'implémentation, les extraits de code les plus significatifs et les résultats obtenus. L'ordre de présentation suit l'ordre de développement effectif, conditionné par les dépendances entre modules.

4.1 Module 1 — Authentification et Gestion des Accès

Ce module constitue le fondement de la sécurité de toute l'application. Son implémentation correcte était une condition sine qua non au développement de tous les autres modules, car chaque vue protégée s'appuie sur le mécanisme d'authentification mis en place ici.

Modèle User personnalisé

Nous avons étendu la classe `AbstractUser` de Django pour ajouter le champ `role` (une clé étrangère vers la classe `Role`). Cette approche nous permet de bénéficier de toutes les fonctionnalités de gestion des utilisateurs déjà implémentées dans Django (hachage des mots de passe, sessions, permissions) tout en y ajoutant notre logique métier spécifique.

```
class Role(models.Model):
    role_name = models.CharField(max_length=50, unique=True)
    description = models.TextField(blank=True)
    def __str__(self): return self.role_name

class User(AbstractUser):
    role = models.ForeignKey(Role, on_delete=models.SET_NULL,
                             null=True, blank=True, related_name="users")
    def __str__(self): return self.username
```

Profil employé

La classe `EmployeeProfile` est reliée à `User` par une relation `OneToOne`. Elle stocke les informations propres à chaque employé (téléphone, salaire, département, date d'embauche) et implémente la méthode de calcul de la paie nette.

```
class EmployeeProfile(models.Model):
    DEPT = [("cuisine", "Cuisine"), ("service", "Service"),
            ("direction", "Direction"), ("stock", "Stock")]
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    salary = models.FloatField(default=0.0)
    hire_date = models.DateField(auto_now_add=True)
    department = models.CharField(max_length=50, choices=DEPT)
```

```
def calculate_payroll(self):
    # Paie nette = salaire brut x 85% (15% de cotisations)
    return round(self.salary * 0.85, 2)
```

Vue de connexion

La vue de connexion utilise la fonction `authenticate()` de Django qui vérifie le couple identifiant/mot de passe de manière sécurisée. Après connexion réussie, l'utilisateur est redirigé vers le tableau de bord.

```
def login_view(request):
    form = LoginForm(request.POST or None)
    if request.method == "POST" and form.is_valid():
        user = authenticate(request,
                            username=form.cleaned_data["username"],
                            password=form.cleaned_data["password"])
        if user and user.is_active:
            login(request, user)
            return redirect("dashboard")
        messages.error(request, "Identifiants invalides.")
    return render(request, "auth/login.html", {"form": form})
```

4.2 Module 2 — Gestion des Recettes

Ce module est techniquement dépendant du Module 3 (Produits) et du Module 5 (Stock), car une recette est nécessairement associée à un plat existant et ses ingrédients correspondent à des articles de stock. Il a donc été développé en parallèle avec le Module 3.

Modèle *Recipe* et *RecipeIngredient*

La classe *Recipe* est reliée à *Product* par une relation *OneToOne* — chaque plat ne peut avoir qu'une seule recette. La classe *RecipeIngredient* modélise les ingrédients avec la quantité requise et l'unité de mesure. La contrainte *unique_together* empêche d'associer deux fois le même article de stock à la même recette.

```
class Recipe(models.Model):
    product = models.OneToOneField(Product, on_delete=models.CASCADE)
    cooking_time = models.IntegerField(default=15) # En minutes
    instructions = models.TextField(blank=True)

    def check_all_ingredients_available(self):
        """Retourne True si TOUS les ingrédients sont disponibles."""
        return all(ri.check_availability()
                   for ri in self.ingredients.select_related("stock").all())

class RecipeIngredient(models.Model):
    recipe = models.ForeignKey(Recipe, on_delete=models.CASCADE,
                              related_name="ingredients")
    stock = models.ForeignKey(Stock, on_delete=models.PROTECT)
    required_qty = models.FloatField()
    unit = models.CharField(max_length=20)
```

```

def check_availability(self):
    return self.stock.quantity >= self.required_qty

def deduct_from_stock(self):
    if self.check_availability():
        self.stock.update_quantity(-self.required_qty)
        return True
    return False

class Meta:
    unique_together = [("recipe", "stock")]

```

4.3 Module 3 — Gestion du Catalogue de Produits

Ce module gère l'ensemble du catalogue des plats proposés par le restaurant. Il est au cœur du système car presque tous les autres modules y font référence : les recettes sont associées aux produits, les commandes se composent de lignes de produits, le stock peut être lié à des produits.

Modèle Product avec historique des prix

Nous avons implémenté un système de traçabilité des prix : chaque modification du prix d'un produit déclenche la création d'un enregistrement PriceHistory avec l'ancienne valeur, la nouvelle valeur et la date du changement. Cela permet au directeur de consulter l'évolution des tarifs dans le temps.

```

class Product(models.Model):
    name = models.CharField(max_length=200)
    price = models.FloatField()
    description = models.TextField(blank=True)
    category = models.ForeignKey(Category, on_delete=models.CASCADE)
    is_available = models.BooleanField(default=True)
    image = models.ImageField(upload_to="products/",
                              blank=True, null=True)

    def toggle_availability(self):
        self.is_available = not self.is_available
        self.save()

```

Vue de création de produit avec validation

Le formulaire ProductForm valide que le prix est strictement positif avant d'accepter la création ou la modification d'un produit. En cas d'échec, un message d'erreur explicite est affiché à l'utilisateur.

```

class ProductForm(forms.ModelForm):
    def clean_price(self):
        price = self.cleaned_data.get("price")
        if price is not None and price <= 0:
            raise forms.ValidationError(

```



```
"Le prix doit être supérieur à 0 FCFA.")
return price
```

4.4 Module 4 — Gestion des Commandes

Ce module est le plus complexe de l'ensemble de l'application, car il coordonne plusieurs sous-systèmes simultanément : la vérification du stock, la mise à jour des quantités, la création des lignes de commande et la notification de la cuisine. La principale difficulté était de garantir l'intégrité des données face aux erreurs possibles.

Transaction atomique pour l'intégrité des données

Nous avons utilisé le décorateur `@transaction.atomic` de Django pour encapsuler l'ensemble de la création d'une commande dans une transaction de base de données. Cela signifie que si une erreur survient à n'importe quelle étape (stock insuffisant, produit supprimé, erreur MySQL), toutes les opérations précédentes sont automatiquement annulées (ROLLBACK), garantissant qu'il ne peut pas exister de commande partiellement créée en base de données.

```
@login_required
@transaction.atomic # ROLLBACK automatique si erreur
def order_create(request):
    if request.method == "POST":
        order = Order.objects.create(
            table_id = request.POST.get("table_id") or None,
            waiter = request.user,
            order_type = request.POST.get("order_type", "onsite")
        )
        for pid, qty in zip(request.POST.getlist("product_id"),
                           request.POST.getlist("quantity")):
            qty = int(qty)
            if qty <= 0: continue
            product = get_object_or_404(Product, id=pid)
            stock = product.stocks.first()
            if stock and stock.quantity < qty:
                # Leve une exception -> ROLLBACK automatique
                raise ValueError(f"Stock insuffisant: {product.name}")
            if stock: stock.update_quantity(-qty)
            OrderLine.objects.create(order=order, product=product,
                                     quantity=qty)

        order.calculate_total()
        order.notify_kitchen() # Après le COMMIT uniquement
        return redirect("order_detail", pk=order.id)
```

Notification cuisine via Signals

La notification vers l'interface cuisine utilise le mécanisme de signaux Django. Nous avons défini un signal personnalisé `order_ready_signal` qui est émis après la validation d'une commande. Ce découplage architectural est important : la logique de création de commande ne dépend pas directement de la logique de notification, ce qui facilite la maintenabilité et les tests.

```
# signals.py
from django.dispatch import Signal, receiver
order_ready_signal = Signal()

@receiver(order_ready_signal)
def notify_kitchen(sender, order, **kwargs):
    print(f"[CUISINE] Commande #{order.id} en attente")
```

Interface cuisine en temps réel

La vue `kitchen_view` affiche les commandes dont le statut est "En attente" ou "En préparation", triées par date de création (les plus anciennes en premier). Le cuisinier peut faire évoluer le statut en un clic, et la table est automatiquement libérée lorsque la commande est marquée comme livrée.

```
def kitchen_view(request):
    orders = (Order.objects
              .filter(status__in=["pending", "preparing"])
              .prefetch_related("lines__product")
              .order_by("date"))
    return render(request, "orders/kitchen.html", {"orders": orders})

def order_update_status(request, pk):
    order = get_object_or_404(Order, pk=pk)
    new_status = request.POST.get("status")
    order.status = new_status; order.save()
    if new_status == "delivered" and order.table:
        order.table.release() # Libération automatique de la table
```

4.5 Module 5 — Gestion du Stock et Inventaire

La gestion du stock est un module critique pour le bon fonctionnement du restaurant. Notre implémentation assure une traçabilité complète de toutes les variations de stock et déclenche des alertes automatiques dès que les niveaux deviennent critiques.

Méthode `update_quantity` avec `StockLog`

Toute modification de la quantité d'un article de stock passe obligatoirement par la méthode `update_quantity()`. Cette méthode centralise la logique de mise à jour, garantit que la quantité ne peut pas devenir négative (`max(0, ...)`), crée automatiquement un enregistrement `StockLog` pour l'historique, et déclenche l'alerte email si le seuil critique est atteint.

```
class Stock(models.Model):
    item_name = models.CharField(max_length=200)
    quantity = models.FloatField(default=0.0)
    unit = models.CharField(max_length=20, choices=UNITS)
    threshold = models.FloatField(default=10.0)
```

```
def update_quantity(self, delta):
    old_qty = self.quantity
    self.quantity = max(0, self.quantity + delta)
    self.save()
    # Historique automatique
    StockLog.objects.create(
        stock=self, old_qty=old_qty,
        new_qty=self.quantity, delta=delta)
    # Alerte si seuil critique atteint
    if self.quantity <= self.threshold:
        self.trigger_alert()
    return self.quantity

def trigger_alert(self):
    from django.core.mail import send_mail
    send_mail(f"Alerte Stock : {self.item_name}",
              f"Quantite critique : {self.quantity} {self.unit}",
              "noreply@rms.cm", ["manager@rms.cm"],
              fail_silently=True)
```

4.6 Module 6 — Gestion des Ressources Humaines

Le module RH étend les fonctionnalités du Module 1 en ajoutant des champs spécifiques à l'EmployeeProfile et en proposant des vues de gestion et de reporting dédiées au responsable RH.

Rapport mensuel par employé

La méthode `get_monthly_report()` calcule dynamiquement le nombre de commandes servies par l'employé dans le mois courant, en interrogeant la table Order. Ce chiffre permet d'évaluer la productivité de chaque serveur.

```
def get_monthly_report(self):
    month = timezone.now().month
    orders = Order.objects.filter(
        waiter=self.user, date__month=month).count()
    return {
        "employee": self.user.get_full_name() or self.user.username,
        "department": self.department,
        "salary": self.salary,
        "net_salary": self.calculate_payroll(),
        "orders_served": orders
    }
```

4.7 Module 7 — Tableau de Bord (Dashboard)

Le tableau de bord est la vitrine analytique de l'application, destinée au Directeur et à l'Administrateur. Il agrège les données de plusieurs modules pour offrir une vue synthétique des performances du restaurant.

Cache Django pour l'optimisation des performances

Sans mécanisme de cache, chaque consultation du dashboard déclencherait plusieurs requêtes SQL complexes (agrégations, sous-requêtes). Nous avons mis en place un cache local Django avec une durée de vie de 5 minutes. Les données sont recalculées uniquement lorsque le cache expire.

```
@login_required
def dashboard(request):
    cache_key = f"dash_{timezone.now().strftime('%Y-%m-%d')}"
    data = cache.get(cache_key)
    if not data:
        today = timezone.now().date()
        orders_today = Order.objects.filter(date__date=today)
        data = {
            "total_today": float(orders_today.aggregate(
                t=Sum("total_amount"))["t"] or 0),
            "orders_count": orders_today.count(),
            "pending": orders_today.filter(status="pending").count(),
            "best_sellers": list(Revenue.get_best_sellers()),
        }
        cache.set(cache_key, data, timeout=300) # 5 minutes
    return render(request, "dashboard/index.html", {
        "raw": data, "data": json.dumps(data, default=str)})
```

Graphiques Chart.js

Les graphiques du dashboard sont rendus côté client avec Chart.js 4. Les données sont transmises depuis Django sous forme de JSON via l'API /dashboard/api/. Cela permet de mettre à jour les graphiques sans rechargement de page. Nous affichons notamment un graphique en barres du chiffre d'affaires mensuel et un classement des plats les plus vendus.

4.8 Fonctionnalité Bonus — Commande Client Publique

Au cours du développement, nous avons constaté qu'il serait pertinent d'offrir aux clients la possibilité de passer commande directement depuis leur téléphone ou leur ordinateur, sans nécessiter de compte. Cette fonctionnalité, non prévue dans le cahier des charges initial, a été ajoutée à l'issue de la phase de développement principale.

La page /order/ est accessible sans authentification. Elle affiche tous les plats disponibles par catégorie avec les quantités ajustables. Le client choisit le type de commande (Sur place, À emporter ou Livraison), ajoute des notes spéciales et valide. Une page de confirmation avec le numéro de commande s'affiche immédiatement après validation.

```
# Pas de @login_required - accessible sans compte
```

```
@transaction.atomic
def public_order_create(request):
    if request.method == "POST":
        order = Order.objects.create(
            waiter=None, # Aucun serveur associe
            order_type=request.POST.get("order_type", "onsite"),
            notes=request.POST.get("notes", "")
        )
        for pid, qty in zip(request.POST.getlist("product_id"),
                           request.POST.getlist("quantity")):
            qty = int(qty)
            if qty <= 0: continue
            product = get_object_or_404(Product, id=pid)
            OrderLine.objects.create(order=order,
                                     product=product, quantity=qty)

        order.calculate_total()
        order.notify_kitchen()
        return redirect("public_order_confirm", pk=order.id)
    categories = Category.objects.prefetch_related("products").all()
    return render(request, "orders/public_order.html",
                  {"categories": categories})
```

Chapitre 5 — Tests, Validation et Difficultés

5.1 Tests manuels effectués

Bien que les tests unitaires automatisés (pytest) soient planifiés pour la Phase 9, nous avons conduit tout au long du développement des tests manuels systématiques pour valider le bon fonctionnement de chaque fonctionnalité avant de passer à la suivante.

Test effectué	URL testée	Résultat	Observateur
Page d'accueil avec boutons Client et Admin	http://127.0.0.1:8000/	Réussi	TONLEU F.R.J.
Connexion avec compte TONLEU_RAMSES	http://127.0.0.1:8000/login/	Réussi	TONLEU F.R.J.
Dashboard avec KPIs et graphique Chart.js	http://127.0.0.1:8000/dashboard/	Réussi	EYEBE S.V.
Création produit avec image uploadée	http://127.0.0.1:8000/products/create/	Réussi	TIA T.B.
Toggle disponibilité produit (masqué du menu)	http://127.0.0.1:8000/products/2/toggle/	Réussi	EYEBE S.V.
Mise à jour prix avec création PriceHistory	http://127.0.0.1:8000/products/1/edit/	Réussi	TONLEU F.R.J.
Création recette avec 5 ingrédients	http://127.0.0.1:8000/products/1/recipe/	Réussi	KENFACK N.R.
Vérification disponibilité ingrédients (false)	Terminal — check_all_ingredients_available()	Retourne False	KENFACK N.R.
Commande sur place avec transaction atomique	http://127.0.0.1:8000/orders/create/	Réussi	TONLEU F.R.J.
ROLLBACK si stock insuffisant (ValueError)	http://127.0.0.1:8000/orders/create/	ROLLBACK OK	TONLEU F.R.J.
Interface cuisine avec commandes en attente	http://127.0.0.1:8000/kitchen/	Réussi	KENFACK N.R.
Libération table après livraison	http://127.0.0.1:8000/orders/1/status/	Table libérée	NGUEKE T.F.P.
Mise à jour stock + StockLog créé automatiquement	http://127.0.0.1:8000/stock/1/update/	Réussi	KENFACK N.R.
Alerte email en mode console (seuil atteint)	Terminal — trigger_alert()	Email affiché	KENFACK N.R.
Commande client publique sans connexion	http://127.0.0.1:8000/order/	Réussi	TIA T.B.
Page confirmation commande publique	http://127.0.0.1:8000/order/confirm/1/	Réussi	TIA T.B.
python manage.py check — 0 erreur 0 warning	Terminal Anaconda	0 issue (0 silenced)	TONLEU F.R.J.

Tableau 5 — Résultats des tests manuels effectués

5.2 Difficultés rencontrées et solutions apportées

Le développement de ce projet a été ponctué de difficultés techniques diverses. Nous les présentons ici de façon transparente, car leur résolution a constitué une partie importante de notre apprentissage.

#	Problème rencontré	Impact	Solution apportée	Temps perdu
1	Oubli du point (.) dans django-admin startproject — structure imbriquée incorrecte	Bloquant	Supprimer le dossier, recréer avec la commande correcte : django-admin startproject resto_groupe7 .	1 heure
2	MySQLdb introuvable sur Windows — Django ne trouve pas le driver MySQL	Bloquant	Ajouter pymysql.install_as_MySQLdb() en toute première ligne de settings.py	2 heures
3	AUTH_USER_MODEL oublié avant première migration — erreurs de migrations en cascade	Bloquant	Supprimer les fichiers de migrations, vider la base, tout remigrer depuis le début	3 heures
4	TemplateSyntaxError sur /orders/create/ — Django parse incorrectement {{% for %}}	Bloquant (page 500)	Remplacer const prices = {{% for %}} par var prices = {{}}; {{% for %}}prices["pk"] = price; {{% endfor %}}	1 heure
5	Bootstrap CDN inaccessible depuis le réseau local — contenu affiché en bas de page	Interface inutilisable	Télécharger Bootstrap localement dans restaurant/static/ + onerror fallback CDN automatique	2 jours
6	Images des produits non servies (erreur 404 sur /media/)	Fonctionnel partiel	Ajouter + static(MEDIA_URL, document_root=MEDIA_ROOT) dans resto_groupe7/urls.py	2 heures
7	Réinstallation Windows — perte de la configuration Anaconda et MySQL	Très bloquant	Reconfiguration complète de l'environnement depuis GitHub (avantage du versionnement Git)	1 journée
8	StockLog non créé — self.save() appelé après StockLog.objects.create()	Fonctionnel partiel	Inverser l'ordre : d'abord self.save(), puis StockLog.objects.create()	1 heure
9	Chart.js graphique vide — données JSON non sérialisables (datetime Django)	Dashboard incomplet	Utiliser json.dumps(data, default=str) pour sérialiser les types datetime	30 minutes

Tableau 6 — Difficultés rencontrées et solutions apportées

Chapitre 6 — Résultats et Présentation de l'Application

6.1 État de l'application

À la date de rédaction de ce rapport, l'application RMS est entièrement fonctionnelle. Elle tourne sans erreur sur le serveur de développement Django et répond à l'ensemble des exigences fonctionnelles définies dans le cahier des charges initial. Le tableau ci-dessous récapitule les vingt-cinq URLs disponibles et leur statut.

URL	Page / Fonctionnalité	Accès requis	Statut HTTP
/	Page d'accueil avec boutons Client et Admin	Public	200 OK
/menu/	Carte des plats du restaurant	Public	200 OK
/order/	Commande client sans connexion	Public	200 OK
/login/	Formulaire de connexion	Public	200 OK
/logout/	Déconnexion et retour à /login/	Connecté	302 Redirect
/dashboard/	Tableau de bord KPIs + Chart.js	Staff	200 OK
/dashboard/api/	API JSON données graphiques	Staff	200 OK
/orders/	Liste de toutes les commandes	Staff	200 OK
/orders/create/	Nouvelle commande (interface staff)	Staff	200 OK
/orders/<pk>/	Détail et facture d'une commande	Staff	200 OK
/orders/<pk>/status/	Mise à jour statut commande	Staff	302 Redirect
/kitchen/	Interface cuisine temps réel	Staff	200 OK
/tables/	Plan et statut des tables	Staff	200 OK
/products/	Catalogue des produits	Staff	200 OK
/products/create/	Ajout d'un nouveau produit	Staff	200 OK
/products/<pk>/edit/	Modification d'un produit	Staff	200 OK
/products/<pk>/toggle/	Activation/désactivation produit	Staff	302 Redirect
/products/<pk>/recipe/	Gestion de la recette d'un plat	Staff	200 OK
/stock/	Inventaire avec alertes	Staff	200 OK
/stock/create/	Ajout d'un article de stock	Staff	200 OK
/stock/<pk>/update/	Mise à jour quantité stock	Staff	200 OK
/stock/<pk>/history/	Historique des variations	Staff	200 OK

URL	Page / Fonctionnalité	Accès requis	Statut HTTP
/hr/	Tableau de bord RH	Staff	200 OK
/hr/<pk>/	Profil détaillé d'un employé	Staff	200 OK
/users/	Gestion des utilisateurs	Staff	200 OK
/admin/	Panneau Django Admin	Superuser	200 OK

Tableau 7 — Liste des 26 URLs de l'application avec leur statut

6.2 Captures d'écran des interfaces

Les captures ci-dessous illustrent les principales interfaces de l'application finale.

Page d'accueil

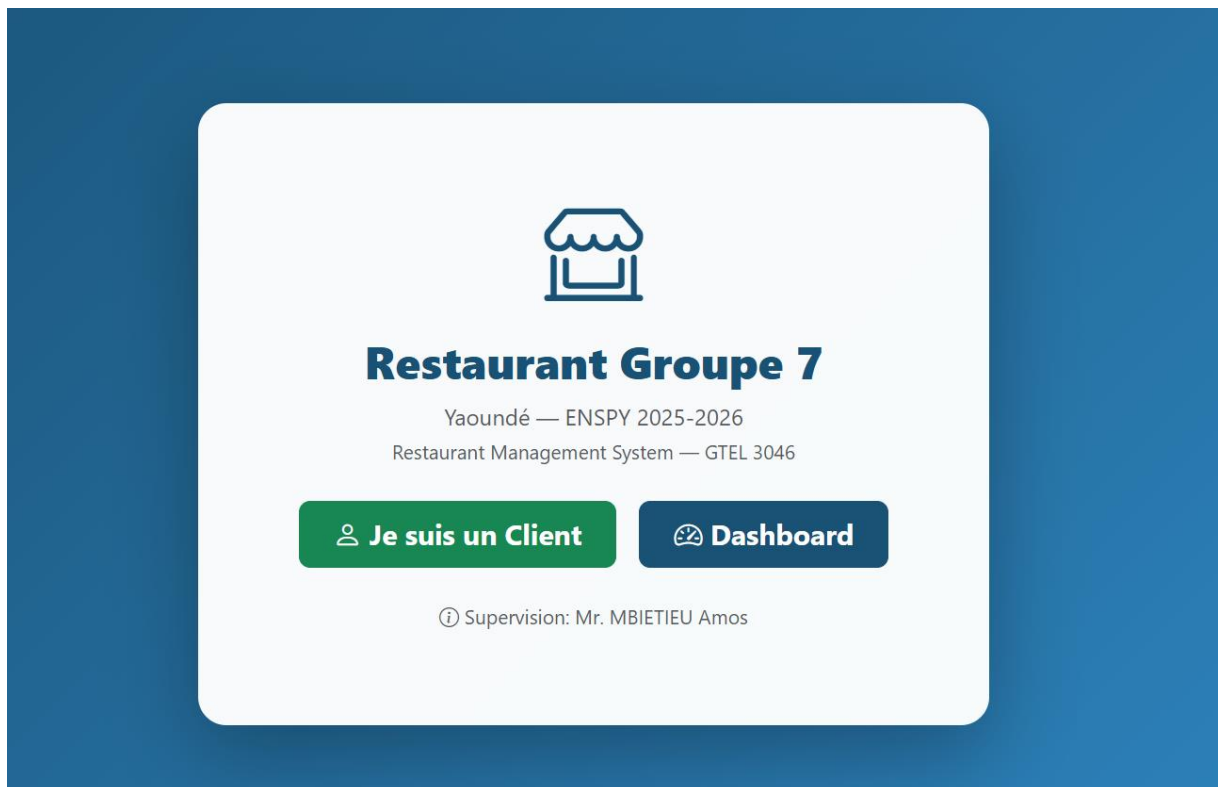


Figure 7 — Page d'accueil : boutons «Je suis un Client» et «Espace Admin»

Tableau de Bord

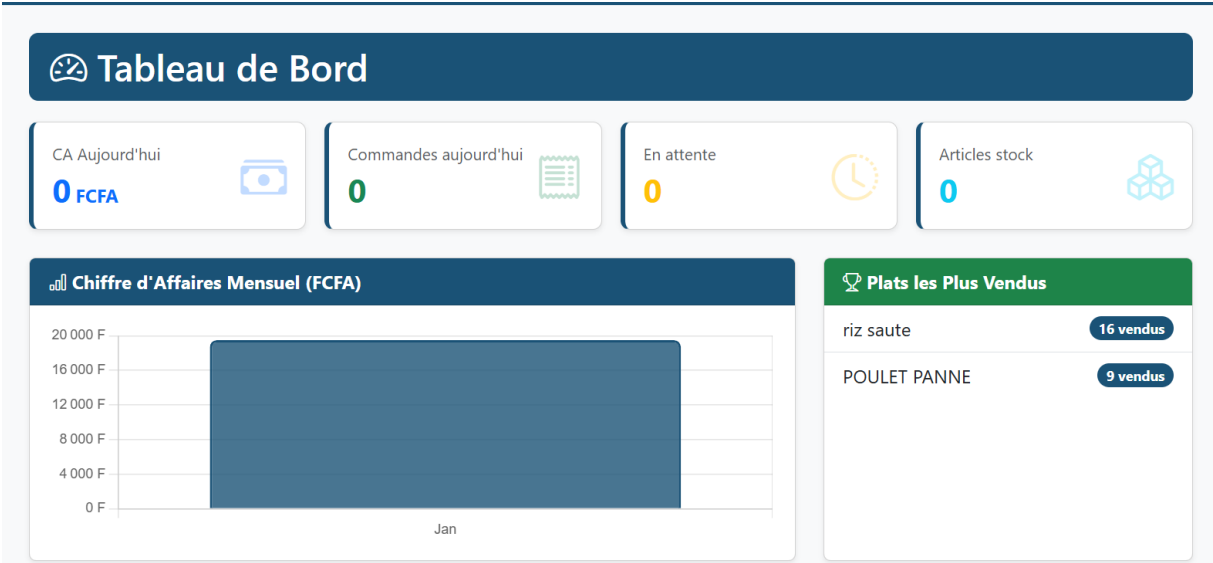


Figure 8 — Tableau de bord : KPI cards, graphique CA mensuel, classement best-sellers

Interface Commandes et Cuisine

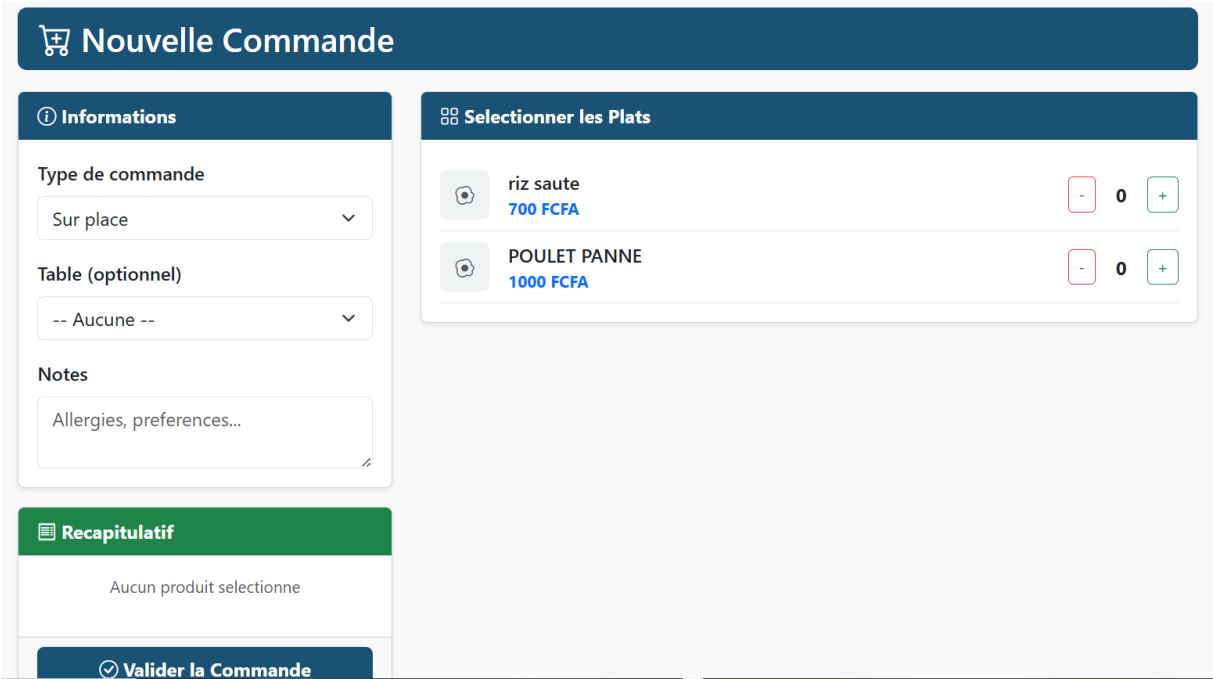


Figure 9 — Interface de prise de commande avec sélection de plats et récapitulatif en temps réel



Figure 10 — Interface cuisine : commandes en attente avec boutons Commencer / Prêt à servir

Commande Client Publique

Toutes les Commandes

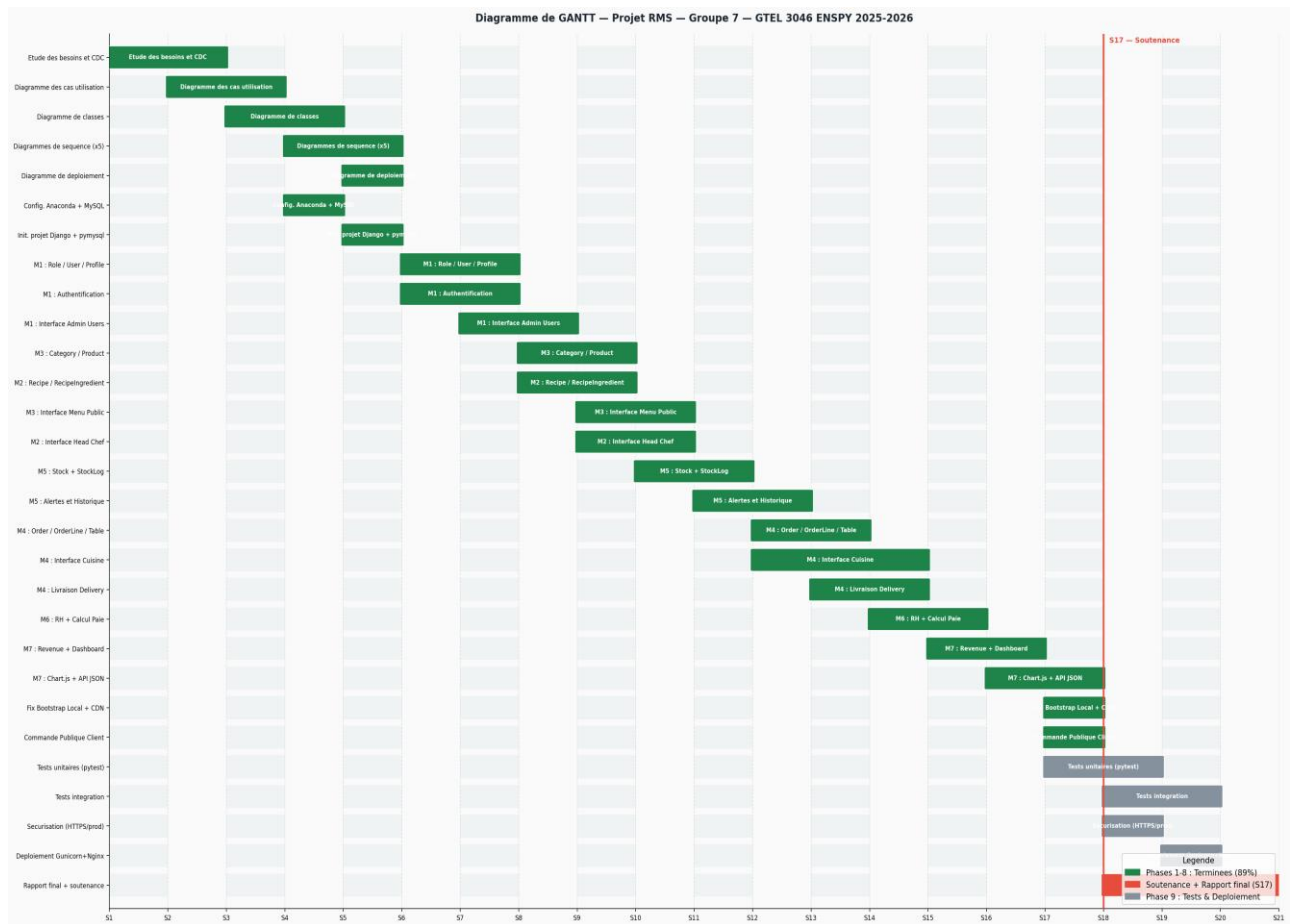
Nouvelle

#	Date	Type	Serveur	Total	Statut	Action
#7	03/06 10:42	A emporter	—	700 FCFA	En attente	
#6	03/06 10:42	Sur place	—	3400 FCFA	En attente	
#5	28/05 13:54	Sur place	TONLEU	2100 FCFA	Paye	
#4	28/05 10:13	Sur place	TONLEU	1400 FCFA	En preparation	
#3	28/05 09:37	Sur place	TONLEU	11900 FCFA	En attente	
#2	27/05 11:28	Livraison	RAMSES	0 FCFA	En preparation	
#1	27/05 11:05	Sur place	TONLEU	0 FCFA	Pret	

Figure 11 — Page de commande publique : accessible sans connexion, choix Sur place / Emporter / Livraison

6.3 Planning GANTT — Avancement du projet

Le diagramme de GANTT ci-dessous présente l'état d'avancement final du projet. En vert figurent les 8 phases entièrement réalisées représentant 89% du projet. La Phase 9 (Tests & Déploiement) est planifiée pour les prochaines semaines.



Phase	Semaines	Modules / Contenu	Avancement
Phase 1 — Modélisation UML	S1 → S5	DCU, Classes, Séquences x5, Déploiement, GANTT	100%
Phase 2 — Configuration Environnement	S3 → S5	Anaconda, MySQL, Django, pymysql, settings.py	100%
Phase 3 — Module 1 Auth	S5 → S7	Role, User, EmployeeProfile, Login/Logout, Admin	100%
Phase 4 — Modules 2 & 3	S7 → S9	Recipe, RecipeIngredient, Product, Category	100%
Phase 5 — Module 5 Stock	S9 → S11	Stock, StockLog, Alertes, Historique	100%
Phase 6 — Module 4 Commandes	S11 → S14	Order, OrderLine, Delivery, Interface Cuisine	100%
Phase 7 — Module 6 RH	S13 → S15	EmployeeProfile étendu, Paie, Interface RH	100%
Phase 8 — Module 7 Dashboard	S14 → S17	Revenue, Chart.js, Cache, API JSON	100%
Phase 9 — Tests & Déploiement	S16 → S20	pytest, Gunicorn, Nginx, HTTPS, Rapport final	En cours

6.4 Dépôt GitHub et versionnement

L'ensemble du code source du projet est versionné sur GitHub depuis le début du développement. Nous avons adopté une organisation par branches : la branche main contient le code stable et déployable, tandis que les branches de développement par module ont été utilisées pour le travail en équipe.

Lien GitHub du projet : <https://github.com/GENIE-LOGICIEL-GROUPE7/PROJET-GENIE-LOGICIEL-2026>

Le dépôt contient l'ensemble des fichiers du projet Django, le fichier requirements.txt listant toutes les dépendances, le script fix_bootstrap.py pour le déploiement Bootstrap local, les diagrammes UML exportés en PNG, ainsi qu'un fichier README.md détaillant les étapes d'installation pour l'évaluateur.

Conclusion Générale et Perspectives

Ce projet de développement du Restaurant Management System (RMS) nous a permis de parcourir l'intégralité du cycle de vie d'un logiciel, de la phase de spécification des besoins jusqu'à la livraison d'une application web fonctionnelle. En équipe de cinq, nous avons su nous organiser, nous répartir les tâches efficacement et surmonter les difficultés techniques qui se sont présentées tout au long du semestre.

Sur le plan technique, ce projet nous a confrontés à des problèmes concrets de développement que les cours théoriques n'avaient pas totalement anticipés : la gestion des dépendances sur Windows, la correction de bugs liés à l'environnement réseau, la résolution d'erreurs de template Django ou encore la mise en place de transactions atomiques pour garantir l'intégrité des données. Chacune de ces difficultés a été une occasion d'approfondir notre compréhension des technologies utilisées.

Sur le plan de la gestion de projet, le diagramme de GANTT que nous avons établi en début de semestre s'est révélé un outil précieux. Il nous a permis de visualiser en permanence notre position dans le planning et d'anticiper les risques de retard. Le versionnement sur GitHub a également joué un rôle crucial, notamment lors de la réinstallation Windows de notre machine principale qui aurait pu entraîner une perte irréversible du code source.

Au terme de ce travail, les huit premières phases sont entièrement réalisées (89% d'avancement global). L'application est opérationnelle, avec un code propre, documenté et versionné. L'ajout de la fonctionnalité de commande client publique (non prévue dans le cahier des charges initial) démontre notre capacité à proposer des améliorations pertinentes en cours de projet.

Perspectives

La dernière phase du projet (Phase 9 — Tests & Déploiement) comprend les activités suivantes, dont la réalisation permettra de compléter le cycle de développement :

1. Écriture des tests unitaires automatisés avec pytest-django pour les modèles, les vues et les formulaires, avec un objectif de couverture de code supérieur à 70%.
2. Tests d'intégration du flux complet : création de commande → déduction stock → notification cuisine → mise à jour statut → libération table.

3. Sécurisation pour la production : activation de `DEBUG=False`, génération d'une `SECRET_KEY` robuste, configuration des `ALLOWED_HOSTS`, activation du HTTPS.
4. Déploiement sur serveur Ubuntu avec Gunicorn (4 workers) comme serveur WSGI et Nginx comme proxy inverse, avec configuration du service systemd pour le démarrage automatique.
5. Rédaction du rapport final complet et préparation de la démonstration pour la soutenance devant Mr. MBIETIEU Amos.

À plus long terme, plusieurs améliorations pourraient enrichir le système : l'ajout d'un module de réservation de tables à l'avance, un système de fidélité pour les clients réguliers, l'intégration d'un paiement en ligne (Mobile Money, Orange Money), ainsi qu'une application mobile compagnon pour le suivi des commandes en temps réel.

Bibliographie

Documentation Officielle

- Django Software Foundation. (2025). Django 5.2 Documentation. <https://docs.djangoproject.com/en/5.2/>
- Oracle Corporation. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- Object Management Group. (2017). OMG Unified Modeling Language (UML) Version 2.5.1. <https://www.omg.org/spec/UML/>
- Bootstrap Team. (2024). Bootstrap 5.3 Components Documentation. <https://getbootstrap.com/docs/5.3/>
- Chart.js Contributors. (2024). Chart.js 4 Documentation. <https://www.chartjs.org/docs/latest/>
- PyMySQL Contributors. (2024). PyMySQL Documentation. <https://pymysql.readthedocs.io/>

Ouvrages de Référence

- Sommerville, I. (2016). Software Engineering (10th edition). Pearson Education Limited. ISBN: 978-1292096131.
- Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd edition). Addison-Wesley Professional.
- Holovaty, A., & Kaplan-Moss, J. (2009). The Definitive Guide to Django: Web Development Done Right. Apress. ISBN: 978-1430219361.
- Garcia-Molina, H., Ullman, J.D., & Widom, J. (2009). Database Systems: The Complete Book (2nd edition). Prentice Hall.

Cours et Supports Pédagogiques

- MBIETIEU Amos. (2025-2026). Supports de cours de Génie Logiciel — GTEL 3046. ENSPY, Université de Yaoundé I, Cameroun.
- Cahier des charges du Projet RMS — Groupe 7. (2026). GTEL 3046, ENSPY.

Ressources en Ligne

- Real Python. (2024). Django Tutorials and Best Practices. <https://realpython.com/tutorials/django/>
- Django Girls. (2024). Django Girls Tutorial. <https://tutorial.djangogirls.org/>
- Stack Overflow. (2024). Questions et réponses Django/Python. <https://stackoverflow.com/questions/tagged/django>
- GitHub du projet — Groupe 7. (2026). Restaurant Management System — GTEL 3046. <https://github.com/GENIE-LOGICIEL-GROUPE7/PROJET-GENIE-LOGICIEL-2026>